

LLMs Integration: Kaggle & HuggingFace

How to Work with LLMs on Kaggle & HuggingFace Platforms

Dr. Elena Nazarenko

Agenda

- Google colab
- Hugging face transformers
- Kaggle integration into google colab

Google colab

- Cloud-based Jupyter notebook environment
- No setup required, access from any device
- Free access to GPUs and TPUs
- Easy sharing and collaboration
- Pre-installed libraries for machine learning

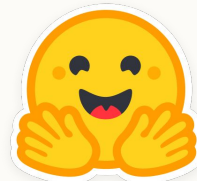
<https://colab.research.google.com/>



GoogleColab Demo:

<https://colab.research.google.com/>





Hugging Face

HuggingFace: Model Hub & Datasets

Vast Model Repository

Access thousands of pre-trained LLMs for various NLP tasks.

Dataset Library

Explore curated datasets for training and evaluation of LLMs.

Community Contributions

Benefit from shared models, datasets, and best practices from AI researchers.

<https://huggingface.co/>



Demo: Hugging Face

<https://huggingface.co/>



Hugging Face and Google Colab integration

```
[ ] from huggingface_hub import notebook_login  
  
notebook_login()
```



Copy a token from [your Hugging Face tokens page](#) and paste it below.

Immediately click login after copying your token or it might be stored in plain text in this notebook file.

Token:

☒ Add token as git credential?

Login

Pro Tip: If you don't already have one, you can create a dedicated 'notebooks' token with 'write' access, that you can then easily reuse for all notebooks.

Hugging Face transformers. Pre-processing

Key Concept: Transformers models don't directly understand raw text, audio, or video. They need the input data to be converted into numerical representations (tokens).

Hugging Face Solution: The **AutoTokenizer** class provides a simple way to pre-process your data.

Pre-processing: AutoTokenizer

Purpose:

Handles the tokenization process, which involves:

- Splitting text into words or subwords.
- Mapping words/subwords to numerical IDs.
- Adding special tokens (e.g., [CLS], [SEP]) required by the model.

Flexibility: AutoTokenizer automatically selects the correct tokenizer based on the model you're using.

Pre-processing: AutoTokenizer

```
# Set DistilBERT tokenizer
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
```

```
# Prepare the text inputs for the model
def preprocess_function(examples):
    return tokenizer(examples["text"], truncation=True)

tokenized_train = small_train_dataset.map(preprocess_function, batched=True)
tokenized_test = small_test_dataset.map(preprocess_function, batched=True)
```

Hugging Face: Pipeline

Pipelines in Hugging Face simplify tasks like text classification, text generation, question answering into a single call.

```
from transformers import pipeline

# Load a sentiment analysis pipeline
classifier = pipeline("sentiment-analysis")

# Classify a sentence
result = classifier("I don't like the new update.")
print(result)
```

Why Pipelines Are Powerful?

- **Abstraction:** You don't need to write code for tokenization, model loading, or output formatting. The pipeline handles it all behind the scenes.
- **Ease of Use:** A simple `pipeline("task-name")` gets you started with state-of-the-art models.
- **Efficiency:** Pipelines are optimized for performance, making them suitable for quick experiments and even production use cases.

Key Components of a Pipeline:

1. **Task Type:** Pipelines are initialized with a specific task (sentiment analysis, text generation, question answering, etc.). This determines what type of model is loaded and how the input/output is handled.
2. **Model:** The pre-trained model used for the task. The model is typically loaded from Hugging Face's model hub.
3. **Tokenizer:** Tokenizes the input text into the format that the model expects.
4. **Framework:** Pipelines can work with either PyTorch or TensorFlow backends.

Pipelines for large variety of tasks:

Hugging Face provides pipelines for a large variety of tasks:

- **Text Processing:** sentiment analysis, text generation, translation, summarization, question answering, named entity recognition, text classification, fill-mask.
- **Audio Processing:** automatic speech recognition (ASR), audio classification.
- **Computer Vision:** image classification, object detection, image segmentation.
- **Multimodal Tasks:** visual question answering, image captioning.

Common Pipelines in Hugging Face: Named Entities Recognition

```
from transformers import pipeline

ner_pipeline = pipeline("ner", aggregation_strategy="simple")

sentence = "The Lucerne School of Computer Science and Information Technology campus is located in Rotkreuz."

entities = ner_pipeline(sentence)

print(entities)
```

```
from transformers import pipeline

# Use a specific pre-trained model for NER
ner_pipeline = pipeline("ner", model="dbmdz/bert-large-cased-finetuned-conll03-english", aggregation_strategy="simple")

sentence = "The Lucerne School of Computer Science and Information Technology campus is located in Rotkreuz."

entities = ner_pipeline(sentence)

print(entities)
```

Bring Your Own Model/Tokenizer

```
tokenizer = AutoTokenizer.from_pretrained("your-tokenizer-id")  
model = AutoModelForSequenceClassification.from_pretrained("your-model-id")  
my_pipeline = pipeline("text-classification", model=model, tokenizer=tokenizer)
```




Kaggle



Kaggle: Data&Models

Models

Discover and use thousands of machine learning models, including the most popular diffusion models and LLMs. [Learn how](#) to share with the community and use the [kagglehub library](#).

+ New Model

Your Work



Search Models

All Filters

All Models

Task

Data Type

Framework

Publisher

Language

Lice

Trending Models

See All

Llama 3.2

Meta

8 Variations · 11 Notebooks

The Meta Llama 3.2 collection of m...

283



Gemma 2 2b JPN IT

Google

3 Variations · 4 Notebooks

Gemma is a family of lightweight, st...

24



Gemma

Google

59 Variations · 318 Notebooks

Gemma is a family of lightweight, o...

6507



AI-QR-Code-Decoder-DeepLearning

Brainydaps

1 Variation · 0 Notebooks

Xception DL architecture trained on...

130



Kaggle: Data & Models

≡ kaggle

+ Create

🏠 Home

🏆 Competitions

📊 Datasets

🔗 Models

> Code

🗨️ Discussions

📖 Learn

✓ More

📁 Your Work

▼ VIEWED

🌐 Gemma

📄 USPTO: KerasNLP Sta...

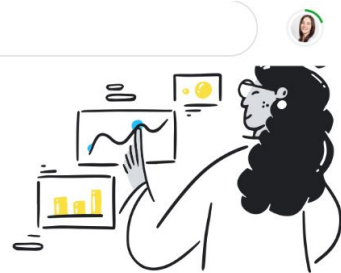
🔍 Search

Datasets

Explore, analyze, and share quality data. [Learn more](#) about data types, creating, and collaborating.

+ New Dataset

Your Work



🔍 Search datasets

≡ Filters

All datasets

Computer Science

Education

Classification

Computer Vision

NLP

Data Visualization

Pre-Trained Model

🌟 Trending Datasets

See All

CUSTOMER CHURN



Telco Customer Churn

Abdallah Wagih Ibrahim · Updated ...



Laptop Dataset

Pradeep Jangir · Updated a month ...



House Price Prediction Dataset



NYC Taxi Fare Dataset

Disha Singh · Updated 11 days ago

Google Colab: Kaggle Integration

Set environment variables for KAGGLE_USERNAME and KAGGLE_KEY.

```
[ ] import os
    from google.colab import userdata

# Note: `userdata.get` is a Colab API. If you're not using Colab, set the env
# vars as appropriate for your system.

os.environ["GITHUB_TOKEN"] = userdata.get('GITHUB_TOKEN')
os.environ["KAGGLE_USERNAME"] = userdata.get('KAGGLE_USERNAME')
os.environ["KAGGLE_KEY"] = userdata.get('KAGGLE_KEY')
```

Google Colab: Kaggle Integration

 File Edit View Insert Runtime Tools Help [Last edited on](#)

Secrets

 Configure your code by storing environment variables, file paths or keys. Values stored here are private, visible only to you and the notebooks that you select.

{x}

Secret name cannot contain spaces.



Notebook access

Name	Value	Actions
------	-------	---------



GITHUB_TOKEN

.....



KAGGLE_KEY

.....



KAGGLE_USERNAM

.....



[+ Add new secret](#)

[Create Gemini API key](#) 

Access your secret keys in Python via:

```
from google.colab import userdata
userdata.get('secretName')
```



GitHub Token

1. **Log in to GitHub:**
 - Go to github.com and log in to your account.
2. **Go to Settings:**
3. **Access Developer Settings:**
 - On the left-hand sidebar of the **Settings** page, scroll down and click on **Developer settings**.
4. **Generate a Personal Access Token:**
 - Under **Developer settings**, click on **Personal access tokens**.
 - You'll see two options:
 - **Tokens (classic)** – this is the more commonly used type.
 - **Fine-grained tokens** – these offer more granular permissions and control.
 - Click on **Tokens (classic)** and then click on **Generate new token**.
5. **Configure the Token:**
 - Provide a **name** for your token to help you remember what it's used for.
 - Set an **expiration date** for security purposes (you can select 30 days, 60 days, 90 days, or no expiration).
 - Choose the **scopes** (permissions) for your token. To work with repositories, check the box for **repo**, which gives you access to all repositories (both public and private).
6. **Generate and Copy the Token:**
 - After selecting the permissions, scroll down and click **Generate token**.
 - **Important:** Once the token is generated, **copy it immediately** as you won't be able to see it again.



Google Colab Demo: Kaggle integration

